CI Brain

Test Intelligence Platform — Claude Code Execution Roadmap

Owner	Soumya Padhi — B.Tech EE, NIT Rourkela
Goal	Resume-shortlisting project for Core SDE / SWE internship roles
Builder	Claude Code
Companion doc	CLAUDE_CODE_PROJECT_RULES.md — read together with this roadmap
Roadmap type	Phase-driven, not date-driven. A phase ends when its checklist passes, not on a schedule.

What this project is

A backend service that plugs into a Python repo's CI and does three things: detects flaky tests statistically, predicts the minimal set of tests to run for a given code change (test impact analysis), and clusters test failures by root cause with a short AI-generated summary. This mirrors internal developer-infrastructure tooling used at large tech companies, and directly covers the patterns found across five real internship JDs (D. E. Shaw, GEP, Salesforce, Cisco, Visa): DSA, SQL, automated testing, CI/CD, distributed/async processing, observability, and AI-tooling literacy.

Target resume line (fill in real numbers after Phase 4 and Phase 7 — never fabricate them)

Built and deployed a CI test-intelligence platform (FastAPI, PostgreSQL, Docker) that reduced test suite runtime by [X]% via coverage-graph-based test impact analysis and auto-detected [N] flaky tests across a benchmarked open-source repo; [Y]% test coverage, GitHub Actions CI/CD.

HUMAN EDIT — add your own notes / decisions here

Space for your own framing — how you'd describe this project in an interview, in your own words.

SCOPE & KEY DECISIONS (LOCKED)

These decisions were made before build and should not be silently changed mid-project. If a phase seems to require reopening one of these, that's a stop-and-ask moment, not a judgment call to make alone.

Language scope	Python only for the MVP (pytest + coverage.py). No multi-language support.
Coverage mapping	Built on top of coverage.py's own execution data — not custom instrumentation. The dependency graph and impact-selection logic on top of that data is the original work.
Benchmark target	A real open-source Python repo, small to medium (roughly 50-200 tests). Not a large framework like Django or Flask — too slow to replay, too much noise.
Flaky/failure data	Real repos won't naturally produce flaky tests or failures on demand. Synthetic flaky tests and synthetic bugs are deliberately seeded into a cloned copy of the target repo. This must be clearly labeled as synthetic evaluation in the final writeup — never presented as if these were organically discovered.
AI/LLM layer	Single-call LLM summarization of failure clusters into a plain-English root-cause hypothesis. Full RAG over failure history is a stretch goal only — cut it first if time or budget runs short.
Deploy target	Docker + a simple free-tier host (Render/Fly.io/Railway) — pick whichever is fastest to wire up, this is not a design decision worth deliberating over.

Model usage guide

Sonnet is the default for this entire project — routine scaffolding, CRUD, tests, CI/CD config, and most bug fixes are well-specified work it handles efficiently. Fable is reserved for the few points where a wrong-but-plausible-looking answer is the real risk — not for volume of code, but for depth of reasoning.

Phase 1 — Scaffold + Ingestion API	Sonnet
Phase 2 — Replay Harness	Sonnet (escalate to Fable only if repo-specific environment issues survive 2 fix attempts)
Phase 3 — Flakiness Detection	Fable for the statistical threshold design decision; Sonnet for implementation
Phase 4 — Test Impact Analysis	Fable for the dependency-graph/selection algorithm design and for validating the benchmark numbers are honest; Sonnet for implementation
Phase 5 — Failure Clustering + LLM Summary	Sonnet (escalate to Fable only if clustering output looks wrong after 2 attempts)
Phase 6 — CI/CD, Docker, Test Suite	Sonnet
Phase 7 — Dashboard + Benchmark Writeup	Sonnet

EXECUTION RULES — SUPPLEMENT TO CLAUDE_CODE_PROJECT_RULES.md

The base rules (phase-driven roadmap, 20-minute subphases, per-phase checklists, token efficiency, minimum human interaction, HUMAN_GUIDE.md maintenance) apply exactly as written in CLAUDE_CODE_PROJECT_RULES.md. Two additions specific to this project:

1. Escalation rule

If self-correction on a bug fails after 2 attempts, stop. Do not keep guessing and burning tokens. Report: what was tried, what's still failing, and the specific decision needed from Soumya.

2. Plain-language check-in rule

When a genuine decision is needed mid-project, ask in plain language — explain the real-world tradeoff, not the technical mechanism. Don't assume familiarity with algorithm or infra terminology. State what changes for the project depending on the answer, then ask a direct question.

Per-phase escalation checkpoints are marked at the end of each phase below.

PHASE 1: PROJECT SCAFFOLD + INGESTION API

Stand up the service skeleton and the schema everything else writes into.

Subphases (each ≤ 20 minutes of Claude Code work — split further if a step looks bigger)

[ ]	Repo scaffold: FastAPI app skeleton, folder structure, dependency setup
[ ]	Postgres schema design: Repo, TestRun, TestCase, TestResult models (SQLAlchemy)
[ ]	Alembic migration setup, connect app to database
[ ]	Ingestion endpoint: POST /runs — accept JUnit XML or coverage.py JSON, parse, store
[ ]	Query endpoints: GET runs / tests / results for a given repo
[ ]	Unit tests for ingestion parsing logic

Phase completion checklist

[ ]	Implementation matches the schema and endpoints listed above
[ ]	Unit tests cover ingestion parsing, including malformed input
[ ]	Ingested data round-trips correctly through the query endpoints
[ ]	No unresolved errors in local run of the full test suite
[ ]	HUMAN_GUIDE.md updated: what the schema stores and why, how to run the API locally
[ ]	Final verification: a real pytest JUnit XML file ingests cleanly end-to-end

Escalation checkpoint

If JUnit XML / coverage.py JSON parsing hits format edge cases that fail twice, stop and report the specific format issue rather than guessing at a parser fix.

HUMAN EDIT — add your own notes / decisions here

Notes, decisions made, or observations from this phase.

PHASE 2: REPLAY HARNESS

Run the target repo's real test suite N times and feed results into Phase 1's API. Everything after this phase depends on it.

Subphases (each ≤ 20 minutes of Claude Code work — split further if a step looks bigger)

[ ]	Pick the target OSS repo (small, pure-Python, existing pytest suite, ~50-200 tests) — HUMAN DECISION: confirm the repo choice before proceeding
[ ]	Script to clone the repo and install its dependencies in an isolated environment
[ ]	Script to run pytest with coverage.py, capturing JUnit XML and coverage data per run
[ ]	Parse coverage.py output into a file-to-test execution map
[ ]	Loop harness: run the suite N times, push each run's results through the Phase 1 ingestion API
[ ]	Spot-check a handful of stored runs against the raw pytest output for correctness

Phase completion checklist

[ ]	Harness successfully clones, installs, and runs the target repo's suite without manual steps
[ ]	N repeated runs are captured and stored via the ingestion API
[ ]	Coverage data is correctly parsed into a file-to-test map for at least one sample file
[ ]	Tests cover the harness's own parsing logic, not just the target repo's tests
[ ]	HUMAN_GUIDE.md updated: which repo was chosen and why, how to re-run the harness
[ ]	Final verification: re-running the harness twice produces consistent, non-corrupted data

Escalation checkpoint

Repo-specific environment issues (dependency conflicts, flaky test discovery, unsupported Python version) are a likely stop point. If unresolved after 2 attempts, ask in plain language whether to keep debugging this repo or switch to a different one.

HUMAN EDIT — add your own notes / decisions here

Notes, decisions made, or observations from this phase.

PHASE 3: FLAKINESS DETECTION

Detect tests that pass/fail inconsistently on identical code, using seeded synthetic flaky tests since real repos won't reliably provide this on demand.

Subphases (each ≤ 20 minutes of Claude Code work — split further if a step looks bigger)

[ ]	Seed 3-5 synthetic flaky tests into a cloned copy of the target repo (e.g. unseeded random, bad time.sleep race, order-dependent state) — clearly commented as intentionally seeded
[ ]	Run the Phase 2 replay harness against the modified repo N times to generate flaky data
[ ]	Statistical detector: compute pass/fail variance per test across identical-commit runs
[ ]	Threshold + quarantine logic — HUMAN DECISION: what variance threshold counts as 'flaky' for this dataset size
[ ]	Report generator: list of flagged tests with a confidence indicator
[ ]	Tests for the detector: confirm it flags the seeded flaky tests and does not flag stable ones

Phase completion checklist

[ ]	Detector correctly identifies all seeded flaky tests in the benchmark run
[ ]	Detector does not false-positive on genuinely stable tests
[ ]	Threshold logic is documented with the reasoning behind the chosen value
[ ]	Tests cover both the statistical logic and the quarantine flagging
[ ]	HUMAN_GUIDE.md updated: how flakiness is detected, what the threshold means, known limitations
[ ]	Final verification: report output is human-readable and matches what was seeded

Escalation checkpoint

Threshold tuning is a judgment call, not a pure implementation detail — flag the chosen threshold and reasoning to Soumya for a sanity check before marking this phase complete.

HUMAN EDIT — add your own notes / decisions here

Notes, decisions made, or observations from this phase.

PHASE 4: TEST IMPACT ANALYSIS

The hardest phase. Build a dependency graph from coverage data and use it to predict the minimal test subset needed for a given code change, then honestly benchmark the savings.

Subphases (each ≤ 20 minutes of Claude Code work — split further if a step looks bigger)

[ ]	Graph builder: aggregate coverage.py data across stored runs into a file-to-tests map
[ ]	Diff parser: given two commits, compute the set of changed files
[ ]	Impact selection: map changed files to affected tests via graph traversal
[ ]	Correctness check — HUMAN DECISION: manually sanity-check the selection against 2-3 known code changes before trusting the algorithm
[ ]	Benchmark harness: measure full-suite runtime vs. selected-subset runtime, same environment, same diff
[ ]	Tests for the graph builder and the selection logic, including edge cases (no changes, whole-file rewrite)

Phase completion checklist

[ ]	Graph correctly maps at least one verified file to its known dependent tests
[ ]	Impact selection produces a strict subset of the full suite for a real sample diff
[ ]	Benchmark numbers are measured on identical hardware/environment for both runs
[ ]	Tests confirm the algorithm doesn't silently drop tests that should be included
[ ]	HUMAN_GUIDE.md updated: how the graph is built, how selection works, benchmark methodology
[ ]	Final verification: the runtime-reduction number is reproducible on a second benchmark run

Escalation checkpoint

If the selection logic looks correct but produces an implausible benchmark number (too good or inconsistent across runs), stop and report rather than adjusting the methodology to force a cleaner-looking result. This number goes on a resume — it has to be true, not just impressive.

HUMAN EDIT — add your own notes / decisions here

Notes, decisions made, or observations from this phase.

PHASE 5: FAILURE CLUSTERING + LLM SUMMARY

Group similar test failures together and generate a short plain-English root-cause hypothesis per cluster, using seeded synthetic bugs to create a realistic failure corpus.

Subphases (each ≤ 20 minutes of Claude Code work — split further if a step looks bigger)

[ ]	Seed 3-5 synthetic bugs into a cloned copy of the repo (e.g. revert a fix, inject a logic error) — clearly commented as intentionally seeded
[ ]	Run the replay harness to collect failure data: stack traces and tracebacks
[ ]	Clustering logic: group failures by traceback/error signature similarity
[ ]	LLM summarizer: one API call per cluster, producing a short plain-English root-cause hypothesis
[ ]	Tests for the clustering logic against the known seeded bugs
[ ]	[Stretch, cut first if time is short] RAG over historical failure clusters

Phase completion checklist

[ ]	Clustering correctly groups failures caused by the same seeded bug
[ ]	Clustering does not merge unrelated failures into one group
[ ]	LLM summary output is coherent and grounded in the actual traceback, not generic filler
[ ]	Tests cover the clustering logic independent of the LLM call
[ ]	HUMAN_GUIDE.md updated: how clustering works, what the LLM layer does and doesn't do
[ ]	Final verification: spot-check 2-3 cluster summaries by hand for accuracy

Escalation checkpoint

If clustering output looks nonsensical after 2 fix attempts, stop and report — this is a candidate to simplify (e.g. exact-match error-type clustering) rather than force a more complex similarity method to work.

HUMAN EDIT — add your own notes / decisions here

Notes, decisions made, or observations from this phase.

PHASE 6: CI/CD, DOCKER, PLATFORM'S OWN TEST SUITE

Make the platform itself deployable and properly tested — this phase is also direct evidence for the 'automated testing' and 'CI/CD' patterns found across every JD.

Subphases (each ≤ 20 minutes of Claude Code work — split further if a step looks bigger)

[ ]	Dockerfile + docker-compose (app + Postgres)
[ ]	GitHub Actions workflow: lint + test on every PR
[ ]	GitHub Actions workflow: build + deploy job — HUMAN DECISION: pick and set up the deploy target (Render/Fly.io/Railway) and any required account/secrets
[ ]	Fill test coverage gaps identified across Phases 1-5
[ ]	Coverage report generation
[ ]	End-to-end integration test: full pipeline run from ingestion through impact analysis

Phase completion checklist

[ ]	Docker build succeeds cleanly from a fresh clone
[ ]	CI workflow runs lint + tests automatically on a PR
[ ]	Deploy workflow successfully deploys to the chosen host
[ ]	Overall test coverage is at a defensible level (documented, not a forced arbitrary number)
[ ]	HUMAN_GUIDE.md updated: how to run CI locally, how deploy works, how to read the coverage report
[ ]	Final verification: a fresh clone can be built, tested, and deployed following only HUMAN_GUIDE.md

Escalation checkpoint

Deploy account setup and secrets are a human step by nature — Claude Code should prepare everything up to that point and clearly state what Soumya needs to do manually.

HUMAN EDIT — add your own notes / decisions here

Notes, decisions made, or observations from this phase.

PHASE 7: DASHBOARD + FINAL BENCHMARK WRITEUP

A small, legible way to see the system's results, plus the honest writeup that turns the benchmark numbers into a defensible resume line.

Subphases (each ≤ 20 minutes of Claude Code work — split further if a step looks bigger)

[ ]	Dashboard scaffold: React app, basic API integration
[ ]	Views: run history, flagged flaky tests, test impact analysis results
[ ]	Simple chart: runtime reduction across benchmark runs
[ ]	Consolidate HUMAN_GUIDE.md into a final, complete pass
[ ]	Benchmark writeup: final numbers, methodology, and clear labeling of the synthetic seeding approach used in Phases 3 and 5 — HUMAN DECISION: Soumya reviews and approves the framing before it goes on a resume or GitHub
[ ]	README polish for GitHub visibility

Phase completion checklist

[ ]	Dashboard renders real data from the deployed API, not mock data
[ ]	All three core views (runs, flaky tests, impact analysis) are functional
[ ]	Benchmark writeup states methodology plainly, including what was synthetic vs. real
[ ]	HUMAN_GUIDE.md is complete and coherent as a single reference document
[ ]	README clearly explains what the project does in the first few lines
[ ]	Final verification: a fresh reader (not Claude Code) can understand the project from the README alone

Escalation checkpoint

No technical escalation expected here — the real checkpoint is Soumya's own review of the final numbers and framing before anything is published.

HUMAN EDIT — add your own notes / decisions here

Notes, decisions made, or observations from this phase.